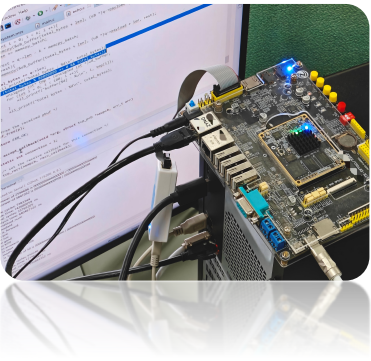




When **FPGA** Meets **Dataflow Analysis**

(An Explorative Step)

Fang Wei, Qinlin Chen, Nairen Zhang, **Jiacai Cui**,
Tian Tan*, Zhiqiang Zuo, and Yue Li*



Dataflow Analysis

- Concept

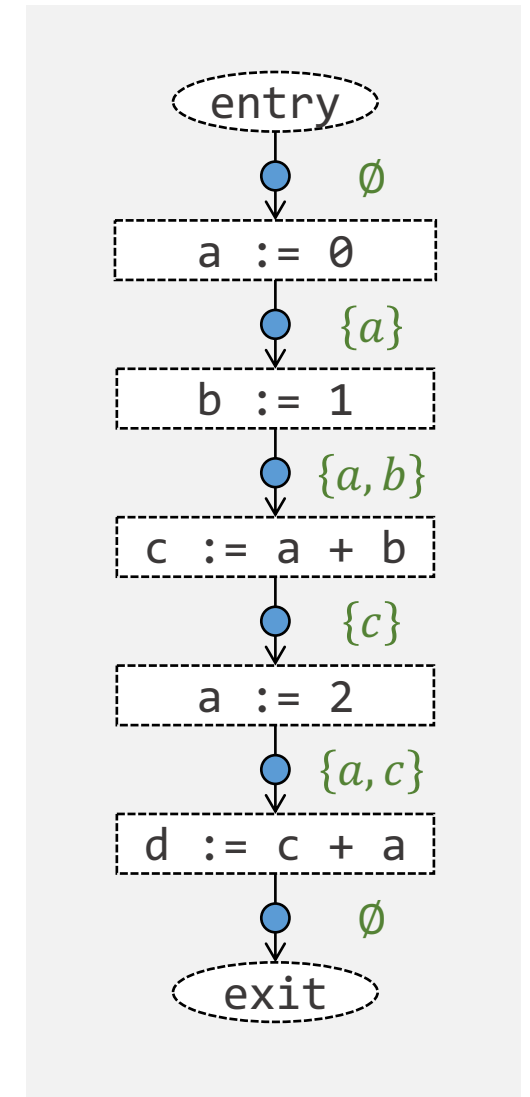
Propagates information along a program's *control-flow graph* to determine **the data fact** at each **program point**.

- Importance

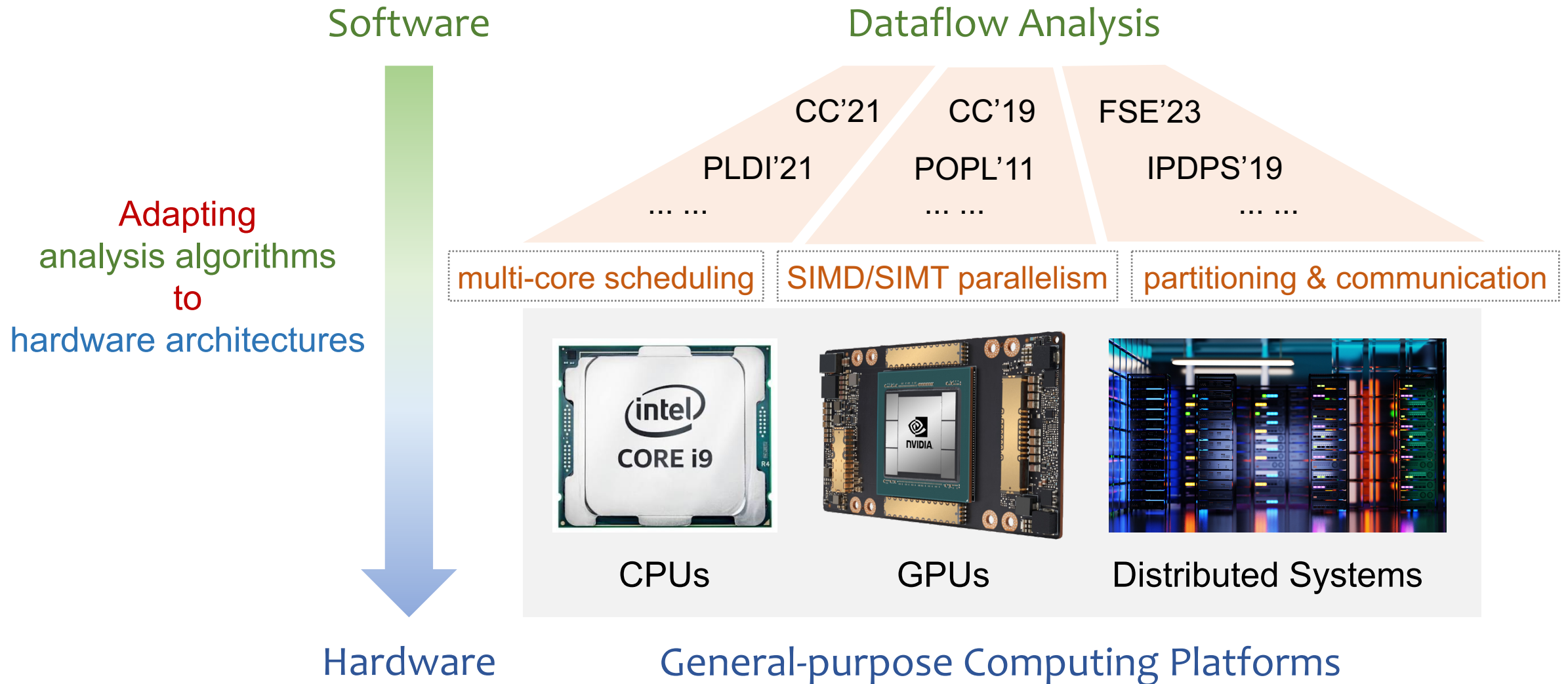
Fundamental in revealing general-purpose program information
*e.g., **live variables**, reaching definitions, available expressions, etc.*

Useful for many software engineering tasks

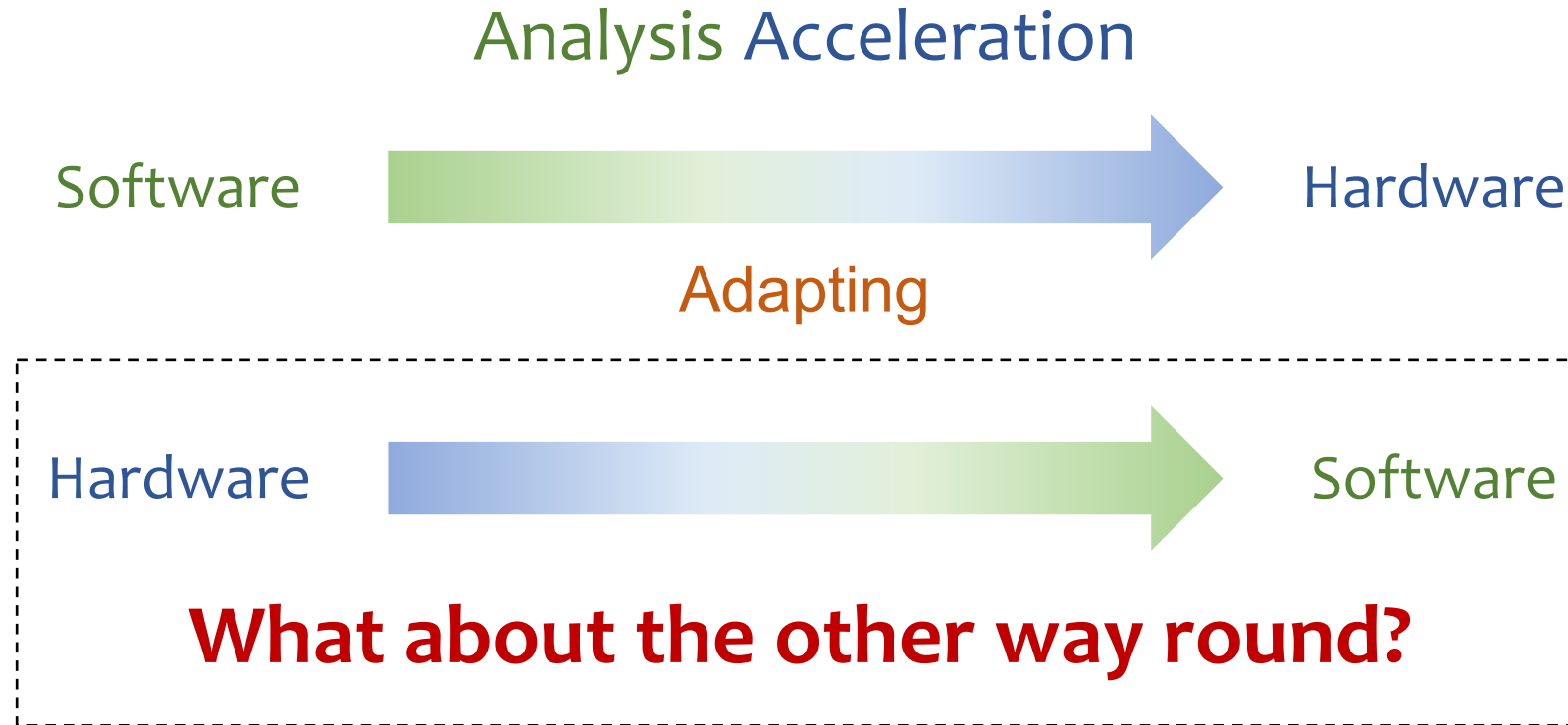
e.g., bug detection, security analysis, code optimization, program understanding, etc.



Analysis Acceleration



Research Question



Designing **specialized hardware** for **dataflow analysis** remains **unexplored**.

Research Question: An Explorative Step

Designing **specialized hardware** for **dataflow analysis** remains **unexplored**.

An FPGA

(Field-Programmable Gate Array)

- Choice 1: FPGA

Highly customizable and well-suited for
rapidly prototyping specialized hardware

Too expensive to tape out an ASIC

Set-based Intra-procedural **Dataflow Analysis**

- Choice 2: Set/Bit-vector based

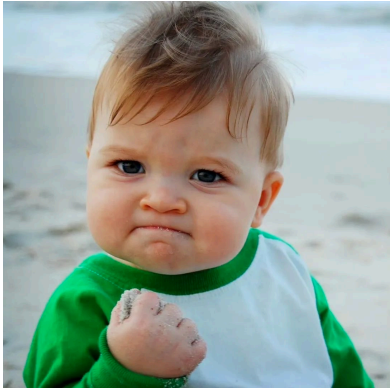
Good **distributivity property** to leverage
$$F(x_1, \dots, x_n)[i] = F(x_1[i], \dots, x_n[i])$$

- Choice 3: Intra-procedural

Easier to implement & widely used

Starting-point Choices

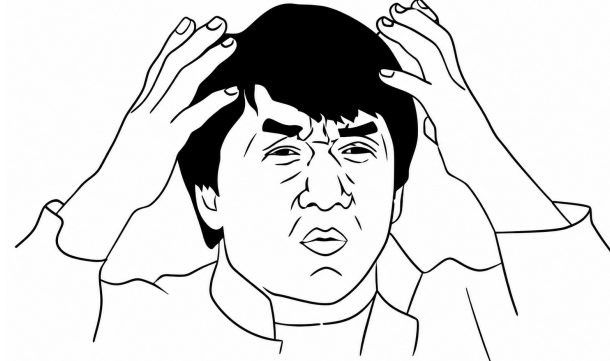
An Explorative Step



Initial Try



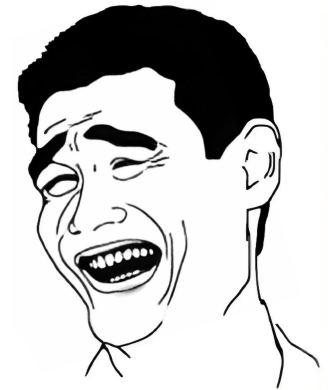
reveal



Challenges

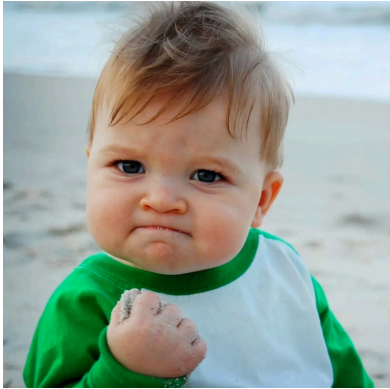


overcome

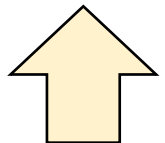


Final Design

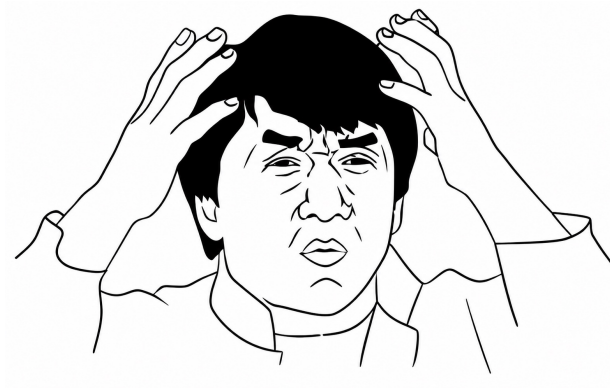
An Explorative Step



Initial Try



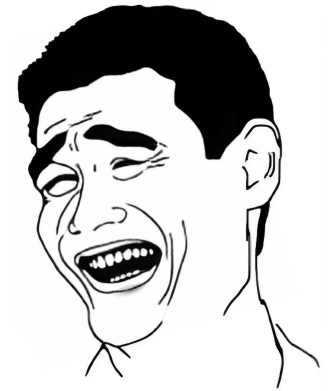
reveal



Challenges



overcome

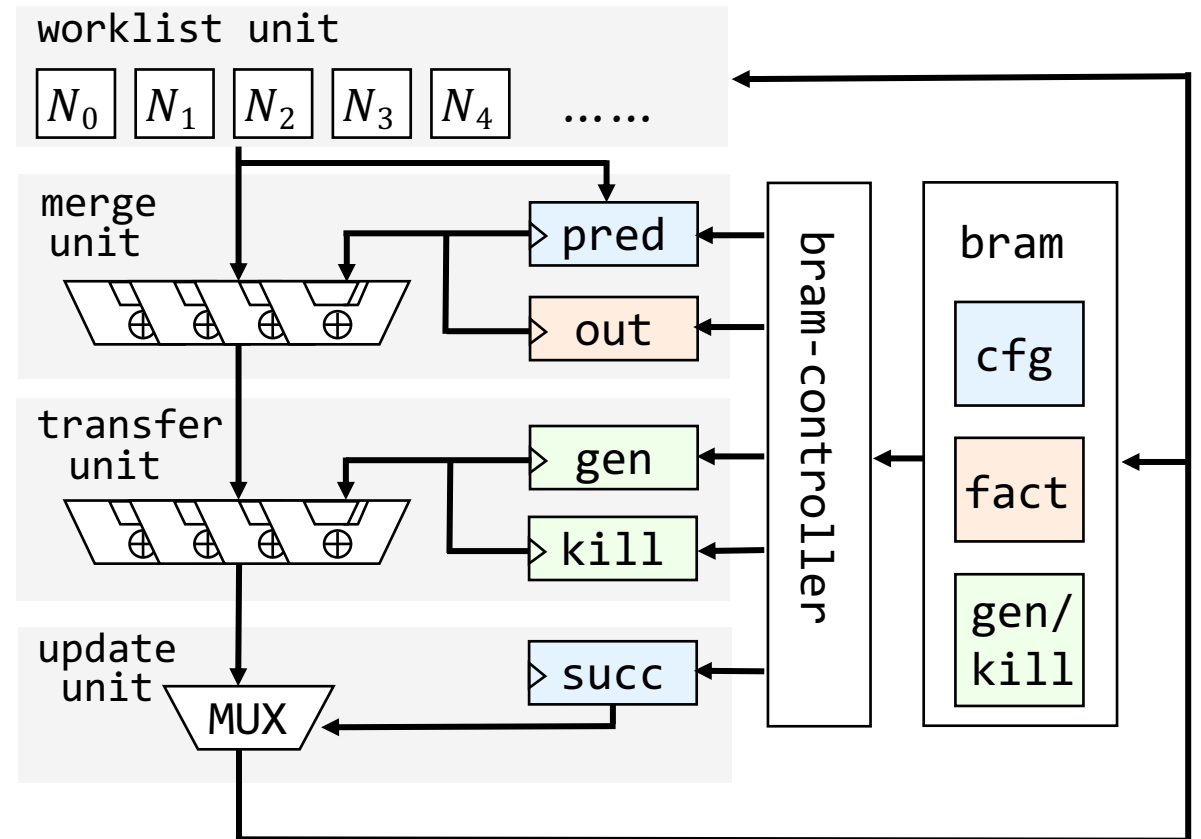


Final Design

Initial Try: A **Straightforward** Accelerator

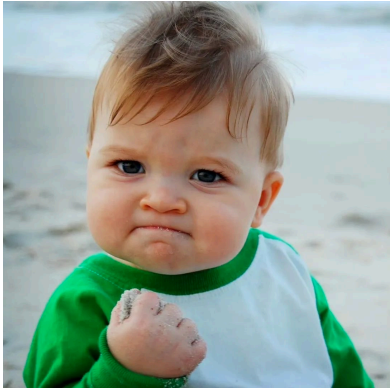
Algorithm 1: Classic Worklist Algorithm

```
1   $W \leftarrow \{\text{all the entry nodes of the CFG}\}$ 
2  repeat
3    Retrieve node  $k$  from  $W$ 
4     $IN[k] \leftarrow \bigotimes_{p \in \text{pred}(k)} OUT[p]$ 
5     $TMP \leftarrow (IN[k] \setminus KILL[k]) \cup GEN[k]$ 
6    if  $TMP \neq OUT[k]$  then
7       $OUT[k] \leftarrow TMP$ 
8       $W \leftarrow W \cup \text{succ}(k)$ 
9  until  $W$  is empty
```



straightforward = simply rewrite the algorithm using an HDL (e.g., Verilog)

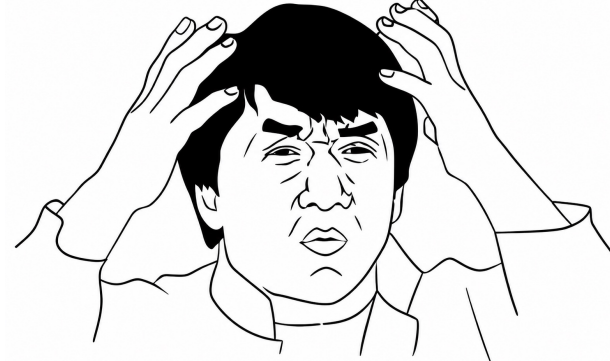
An Explorative Step



Initial Try



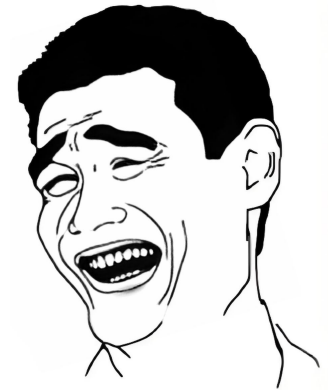
reveal



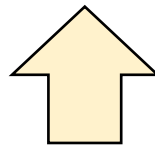
Challenges



overcome



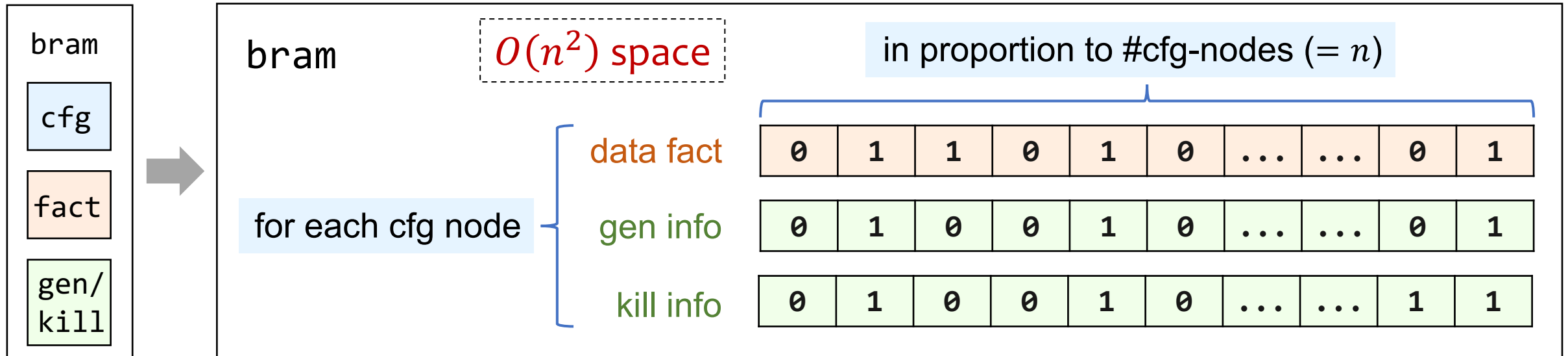
Final Design



Challenges Revealed by the **Straightforward** Design

- **Spatial Infeasibility**

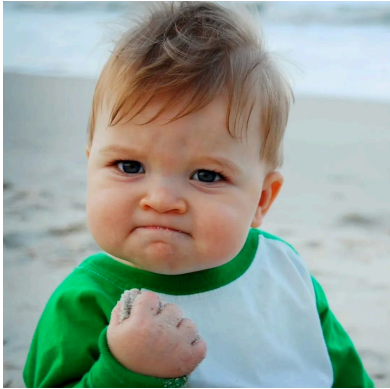
fails to analyze large procedures due to *limited high-speed on-chip memory*



- **Temporal Inefficiency**

fails to outperform high-end CPUs due to the *FPGA's lower clock frequency*

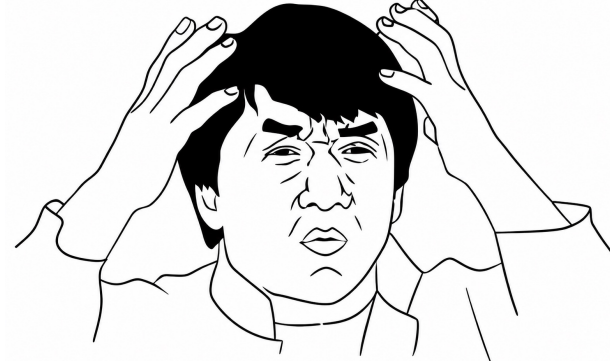
An Explorative Step



Initial Try



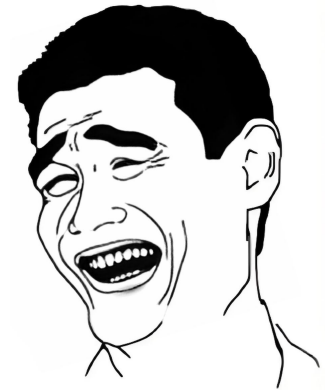
reveal



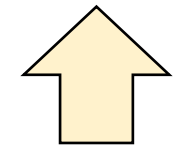
Challenges



overcome

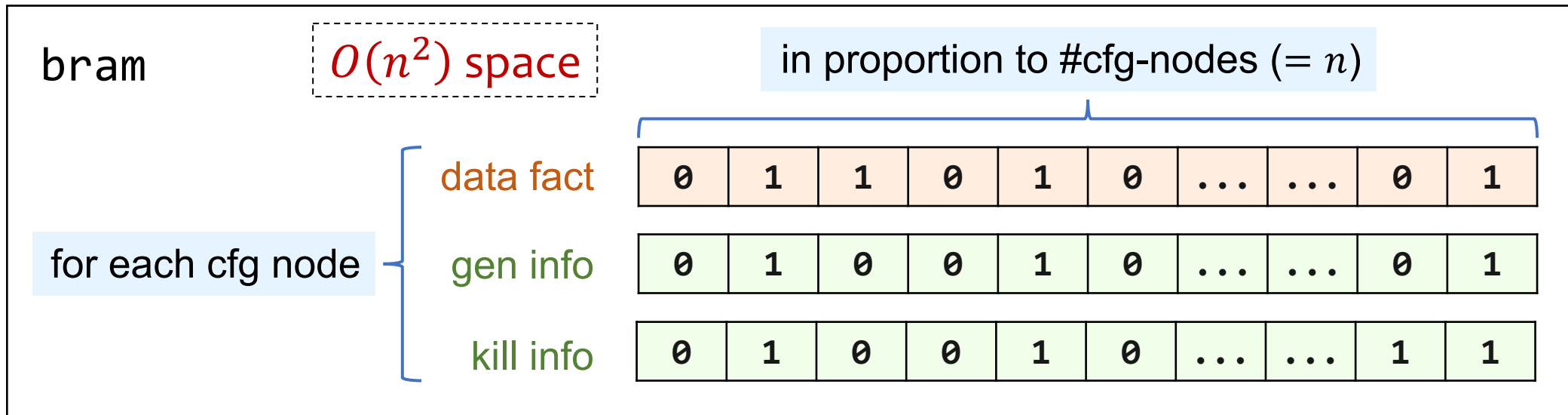


Final Design



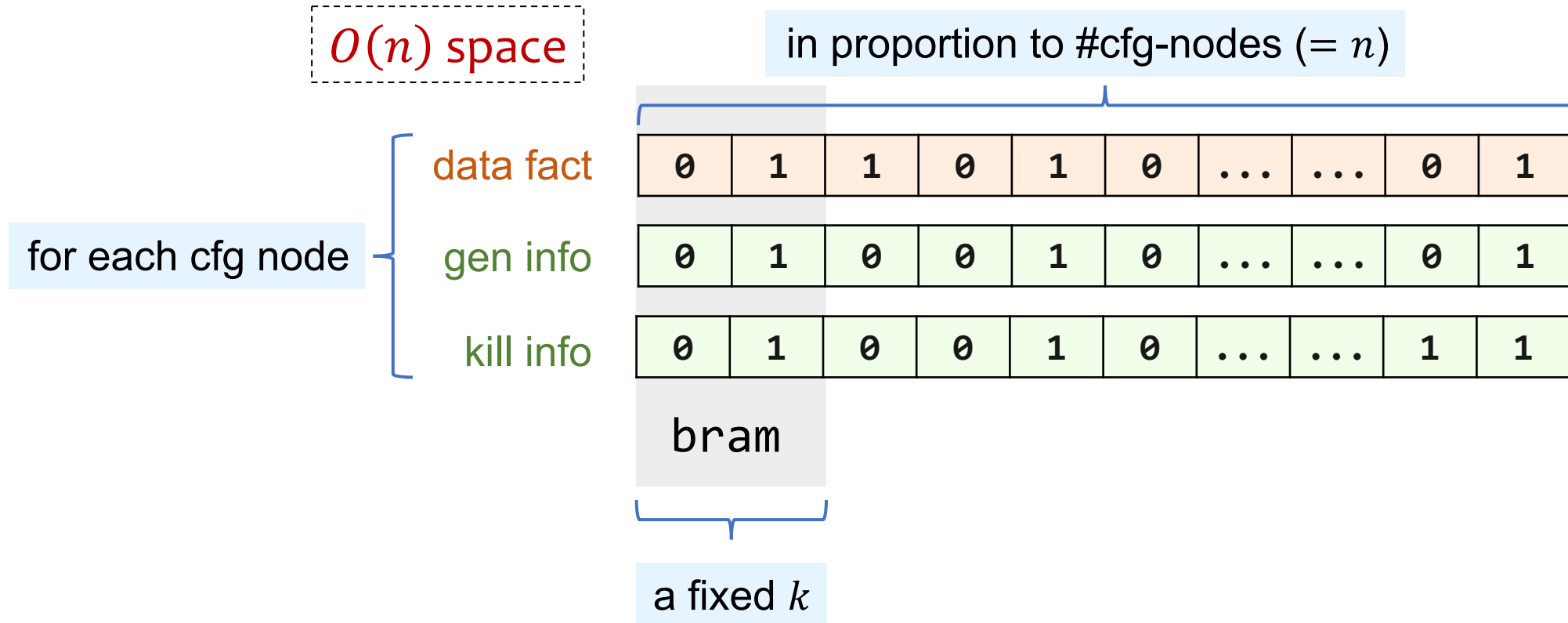
Overcome **Spatial Infeasibility** via **Distributivity**

$$F(x_1, \dots, x_n)[i] = F(x_1[i], \dots, x_n[i])$$



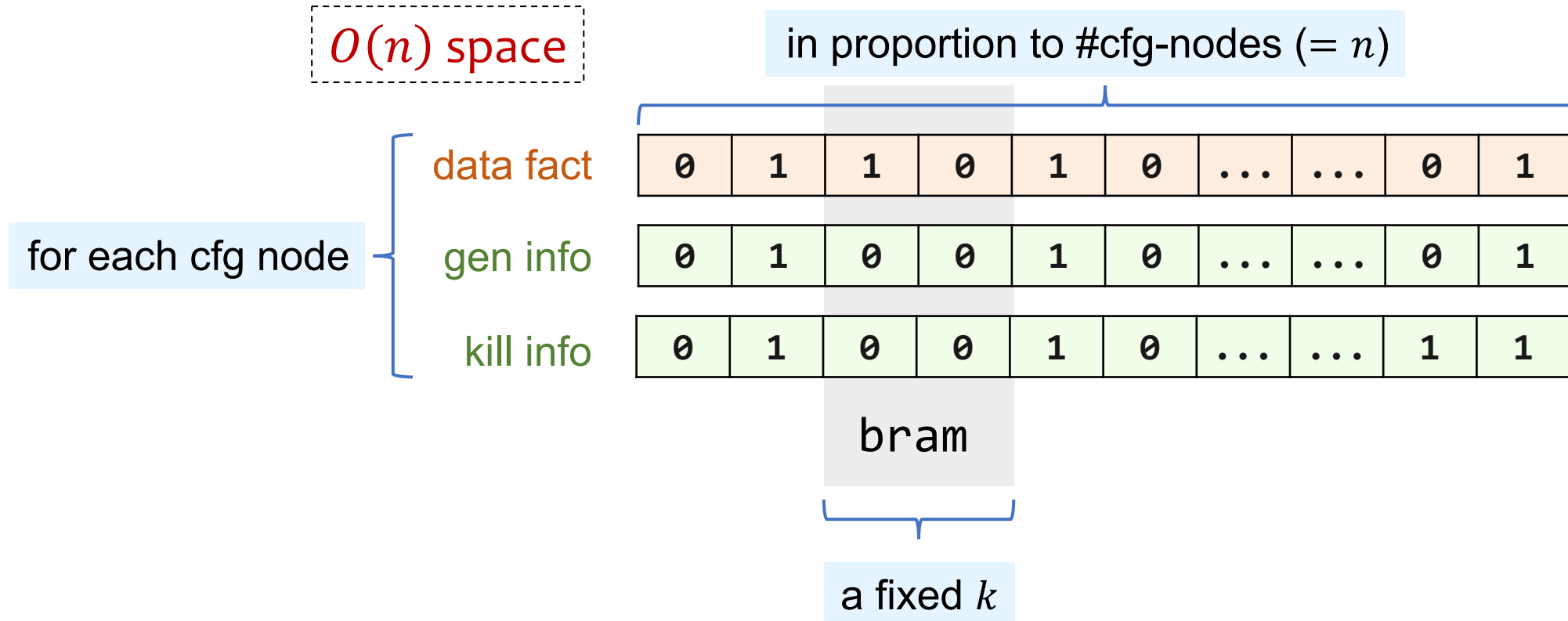
Overcome Spatial Infeasibility via Distributivity

$$F(x_1, \dots, x_n)[i] = F(x_1[i], \dots, x_n[i])$$



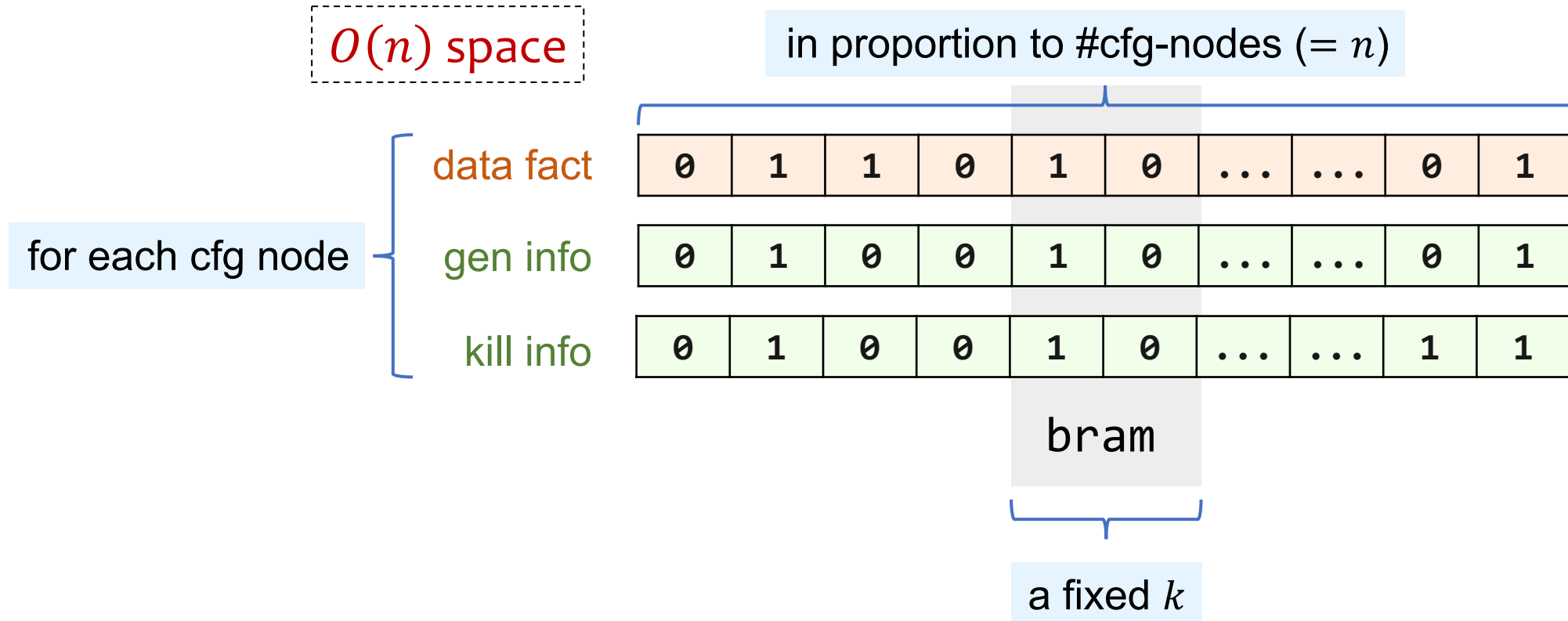
Overcome Spatial Infeasibility via Distributivity

$$F(x_1, \dots, x_n)[i] = F(x_1[i], \dots, x_n[i])$$



Overcome Spatial Infeasibility via Distributivity

$$F(x_1, \dots, x_n)[i] = F(x_1[i], \dots, x_n[i])$$

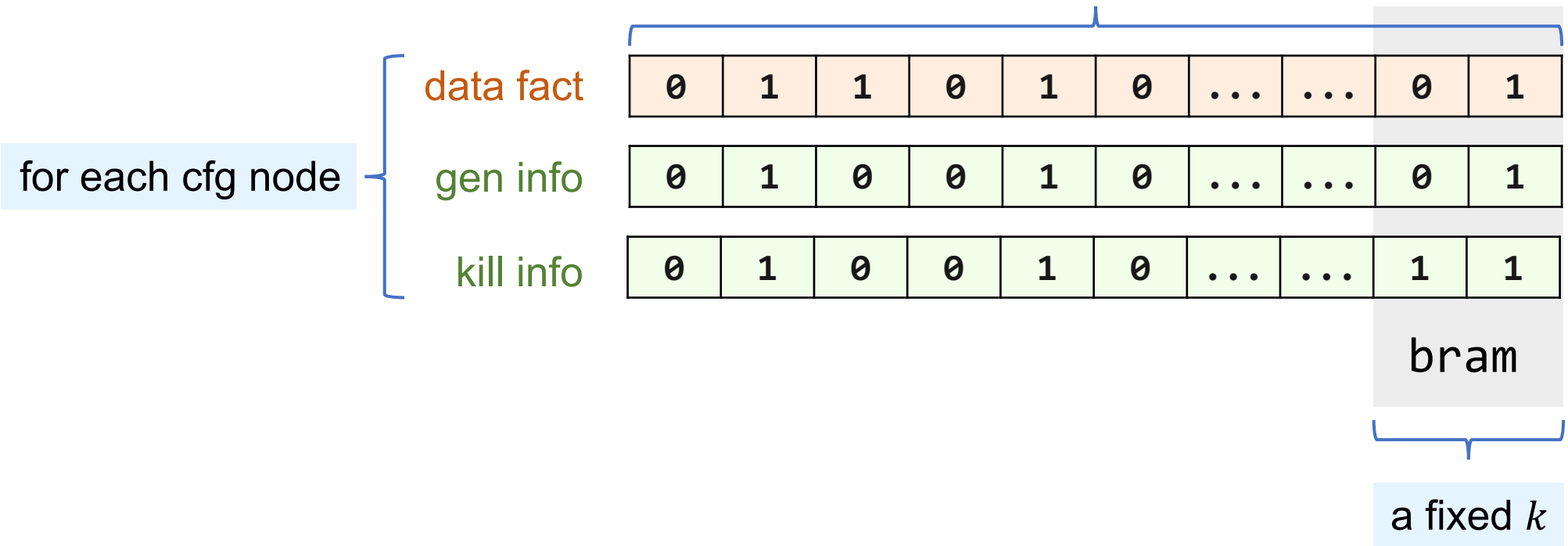


Overcome Spatial Infeasibility via Distributivity

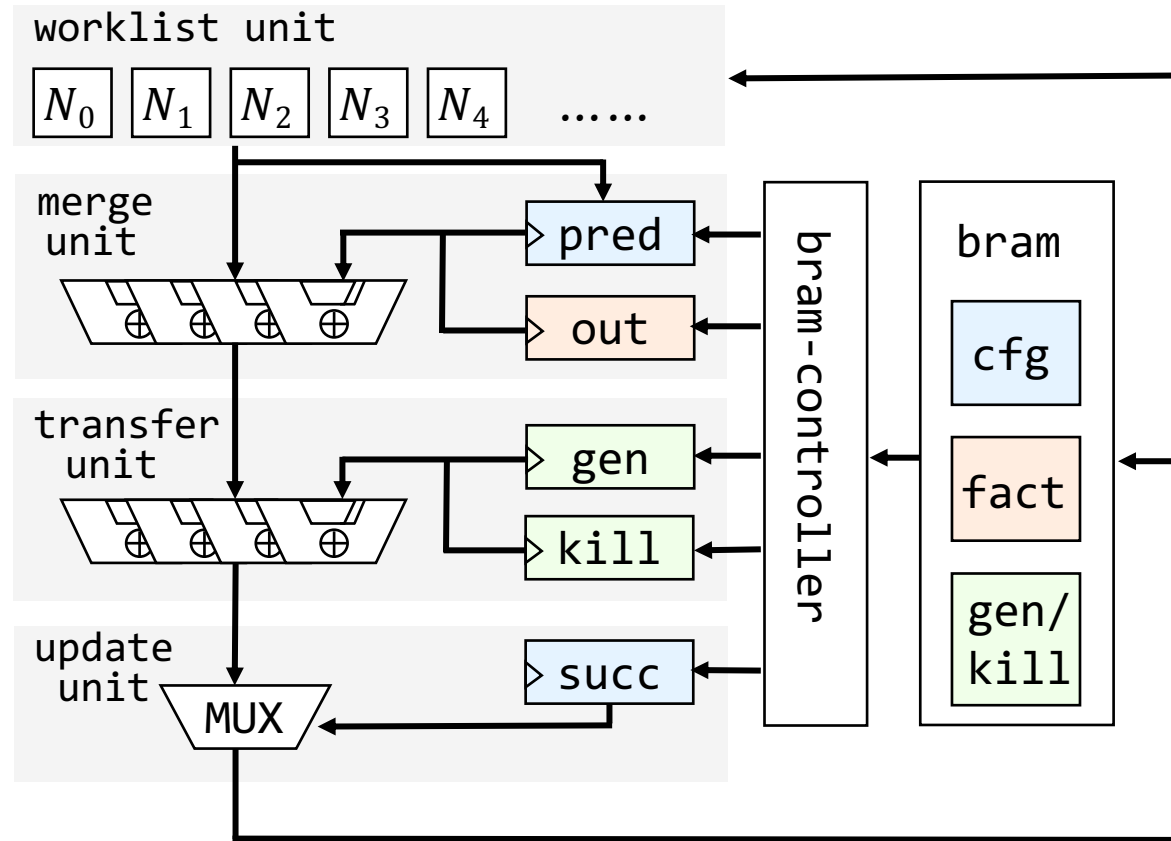
$$F(x_1, \dots, x_n)[i] = F(x_1[i], \dots, x_n[i])$$

$O(n)$ space $\times O\left(\frac{n}{k}\right)$ time

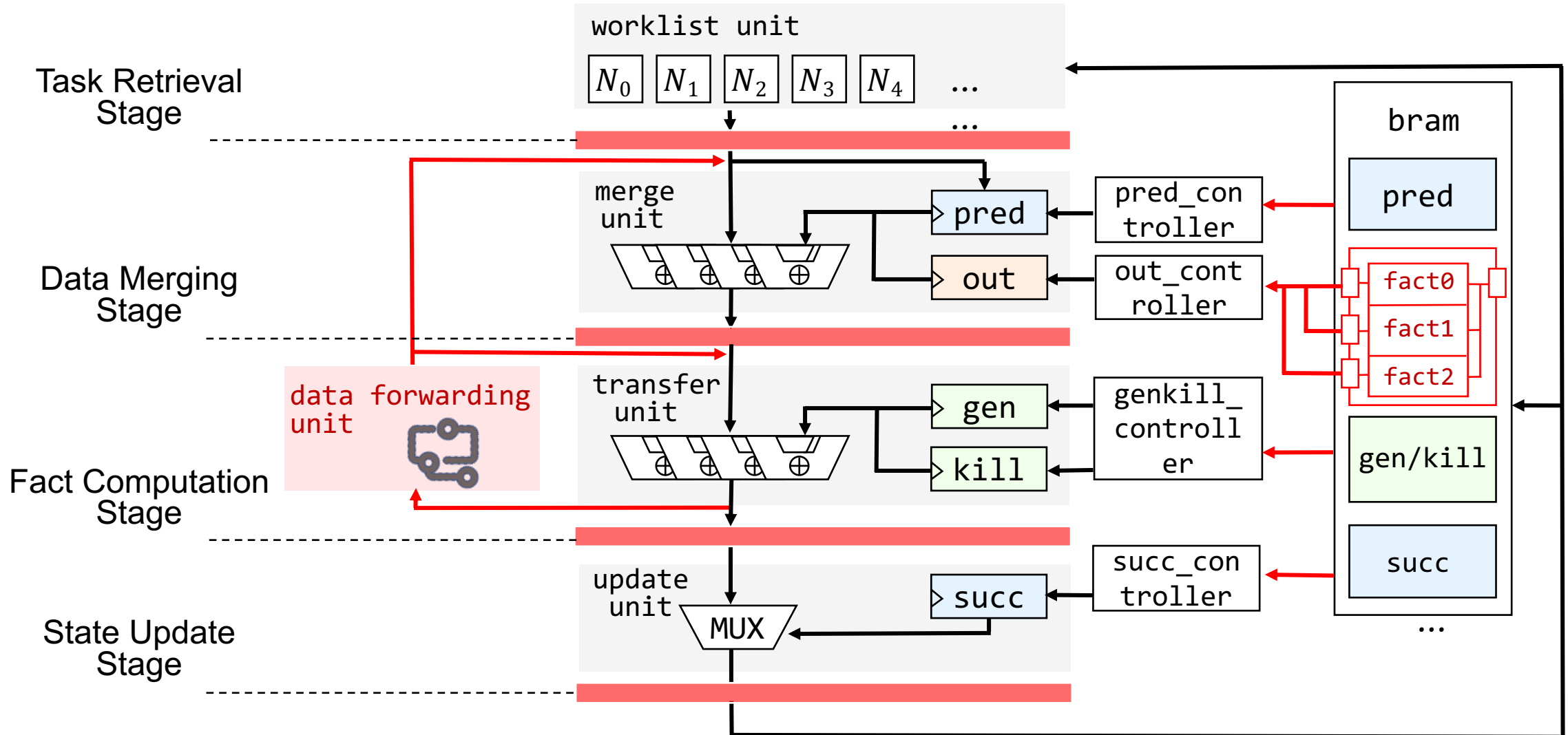
in proportion to #cfg-nodes ($= n$)



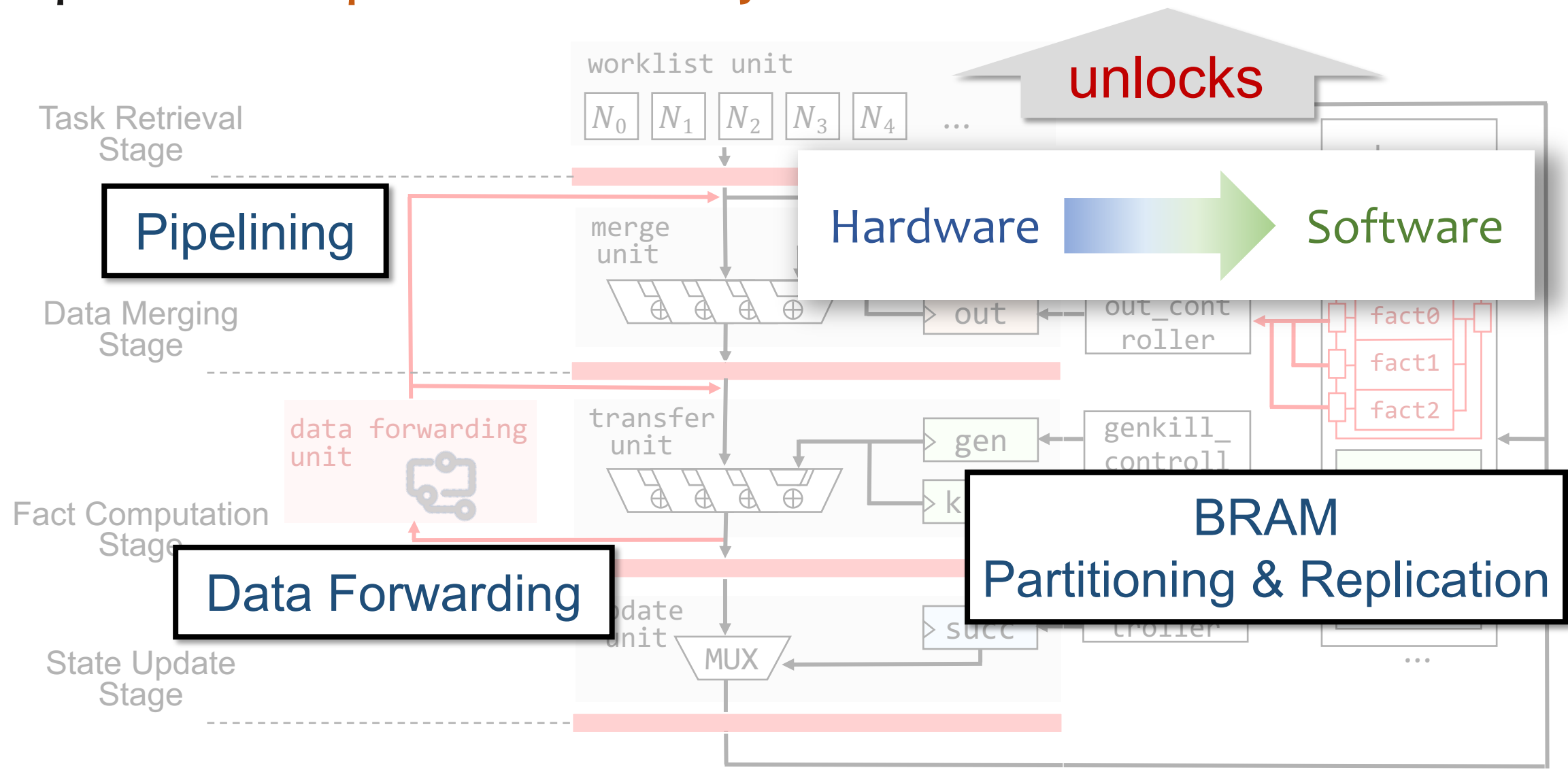
Improve Temporal Efficiency via Hardware Parallelism



Improve Temporal Efficiency via Hardware Parallelism



Improve Temporal Efficiency via Hardware Parallelism





FPGAFlow

(Implementation of our design)

Xilinx Zynq-7020 FPGA (55 MHz)

Evaluation - Setup



FPGAFlow

(Implementation of our design)

Xilinx Zynq-7020 FPGA (**55 MHz**)

vs.

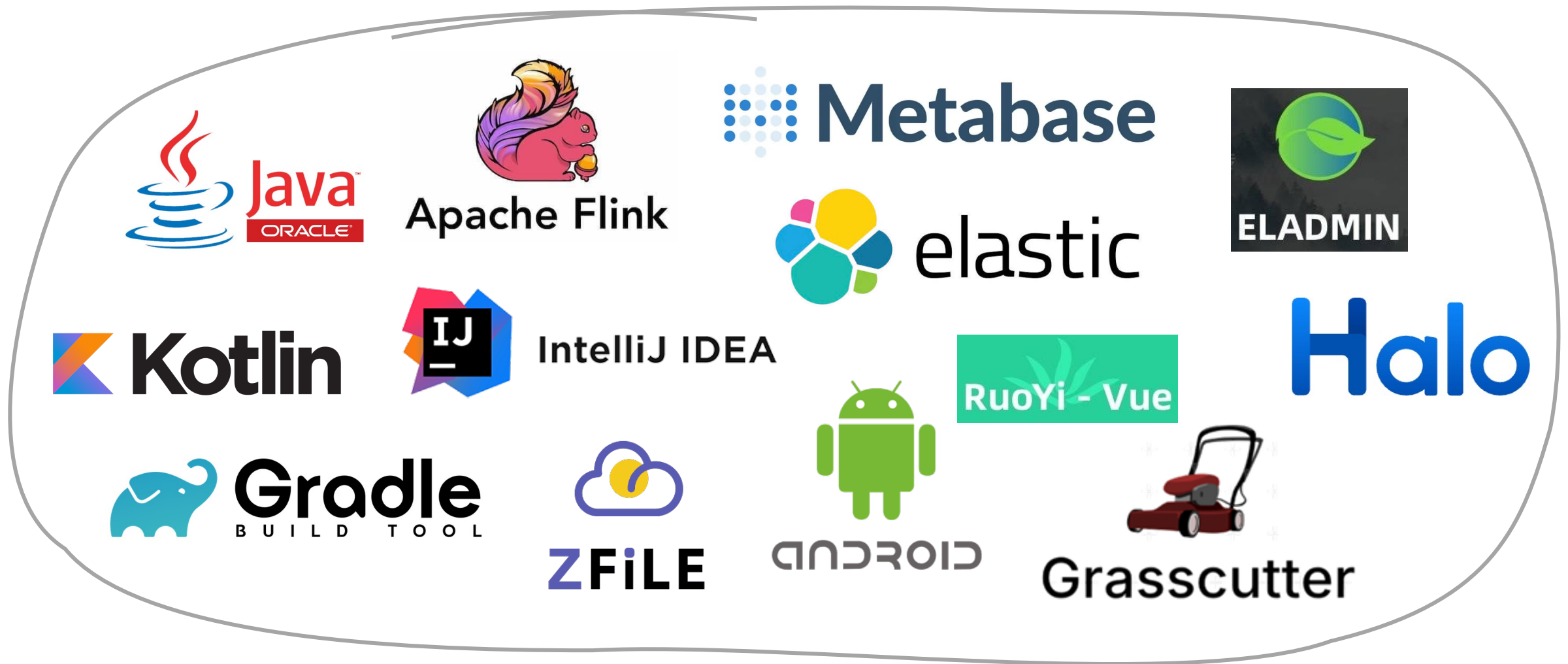


Tai-e

(A state-of-the-art Java static analyzer)

Intel Xeon Silver 4114 CPU (**2.20–3.00 GHz**)

Evaluation - Programs Under Analysis



Real-world Java Projects (Averaged 32.5K Github Stars)

Evaluation - Results

FPGAFlow on a Xilinx Zynq-7020 FPGA (**55 MHz**)

vs.

Tai-e on an Intel Xeon Silver 4114 CPU (**2.20–3.00 GHz**)

- Dataflow Analyses: **Live Variables & Reaching Definitions**
- **Correctness**

Produces *exactly the same analysis results*, amounting to *75 billion bits*.

- **Performance**

A *modest FPGA* outperforms a *server-grade CPU* with an averaged

15.45x speedup for live variables and *12.32x speedup for reaching definitions*.

Conclusion

- **Exploration**

Design **the first specialized hardware for static analysis.**

*We start with **FPGA** and **dataflow analysis**.*

- **Vision**

Accelerate static analyses by plugging an accelerator card into the host machine.

Success of such a paradigm has already been demonstrated in many fields like graphics, deep learning, simulation, bioinformatics, image processing, etc.

